

MANUALE TECNICO: CINEMAX

Indice dei Contenuti

1. **Introduzione e Requisiti di Sistema** pag. 1
 - Panoramica del software CineMax e requisiti di esecuzione (JRE, OS, File System).
2. **Librerie Standard e Dipendenze** pag. 1
 - Analisi delle librerie java.io, java.util e java.time integrate nel progetto.
3. **Architettura del Sistema (Pattern Model-Manager)** pag. 2
 - Approfondimento sul design pattern utilizzato e suddivisione dei layer (UI, Service, Model).
4. **Analisi Dettagliata dei Modelli Dati (Entity)** pag.2
 - Schema delle entità Utente, Proiezione e Prenotazione (POJO e attributi).
5. **Logica di Business: I Gestori (Core Classes)** pag. 3
 - Descrizione operativa dei Gestori (Proiezioni, Utenti, Prenotazioni) e loro interazioni.
6. **Gestione della Persistenza e Operazioni di I/O** pag. 4
 - Parsing dei file CSV, approccio in scrittura (Append) e tecniche di modifica (Copy-Swap).
7. **Gestione delle Eccezioni e Robustezza** pag. 4
 - Gestione errori (try-catch), prevenzione crash e test di robustezza.
8. **Analisi Algoritmica e Strutture Dati** pag. 5
 - Complessità temporale/spaziale e ottimizzazione delle performance di sistema.

1. Introduzione e Requisiti di Sistema

Il progetto **CineMax** è un'applicazione software scritta interamente in linguaggio Java, progettata per amministrare il flusso informativo di un cinema. Il sistema copre l'intero ciclo di vita dell'interazione utente: dalla registrazione, alla consultazione del palinsesto cinematografico, fino alla transazione per la prenotazione dei posti a sedere in sala.

A differenza di soluzioni basate su framework grafici (es. Swing o JavaFX), CineMax adotta un'interfaccia a riga di comando (**CLI - Command Line Interface**). Questa scelta progettuale pone il focus sull'efficienza computazionale, sulla velocità di esecuzione e sulla corretta gestione degli stream di I/O su disco, riducendo l'overhead grafico.

Requisiti di Sistema:

- **Java Runtime Environment (JRE):** Versione 8 o superiore.
- **Sistema Operativo:** Cross-platform. L'utilizzo degli stream di standard input/output (System.in, System.out) garantisce la portabilità su Windows, macOS e distribuzioni Linux.
- **File System:** Permessi di lettura e scrittura nella directory di esecuzione per la gestione della cartella contenente i file .csv.

2. Librerie Standard e Dipendenze

Il progetto si appoggia nativamente al Java Development Kit (JDK), senza richiedere l'iniezione di dipendenze esterne (come Maven o Gradle) per il parsing dei dati.

- **java.io.* (Persistenza):** Vengono utilizzate classi come File, BufferedReader, BufferedWriter, FileReader e FileWriter per la manipolazione dei database in formato CSV. Questo garantisce un controllo a basso livello sulla formattazione dei dati.
- **java.util.* (Strutture Dati e Input):**
 - Scanner: Impiegato all'interno di CineMax.java per catturare e validare in modo robusto gli input testuali e numerici dell'utente.
 - List, ArrayList o LinkedList: Utilizzate dai "Gestori" per mantenere lo stato in memoria (cache locale) durante l'esecuzione dell'applicazione.
- **java.time.* (Gestione Temporale):** Cruciale per la classe Proiezione.java, l'API Date-Time introdotta in Java 8 (es. LocalDateTime, DateTimeFormatter) gestisce il palinsesto, permettendo confronti cronologici

3. Architettura del Sistema (Pattern Model-Manager)

CineMax adotta una variante del pattern architetturale MVC (Model-View-Controller), specificamente orientata ai servizi: il pattern **Model-Manager**.

Il codice è rigorosamente segmentato per responsabilità:

1. **View/UI (Interfaccia Utente):** La classe CineMax.java funge da Entry Point (contiene il main). Gestisce i loop di menu, la stampa a video e l'acquisizione dell'input, agendo da ponte tra l'utente e la logica.
2. **Manager (Logica/Service Layer):** Le classi Gestore... (Utenti, Proiezioni, Prenotazioni) incapsulano le regole di business. Sono responsabili della validazione, della comunicazione tra oggetti e dell'interfacciamento con i file CSV.
3. **Model (Entità):** Le classi Utente, Proiezione e Prenotazione sono Plain Old Java Objects (POJO). Sono "anemiche", ovvero contengono solo attributi, costruttori, getter e setter, senza logica di accesso al disco.

Questa separazione garantisce un'alta manutenibilità: se si decidesse di passare da un CSV a un database SQL, basterebbe modificare i soli Gestore, lasciando intatti l'interfaccia UI e i Modelli.

4. Analisi Dettagliata dei Modelli Dati (Entity)

Basandosi sull'architettura estratta, i modelli definiscono lo schema dei dati in memoria:

- **Utente.java:** Rappresenta l'attore del sistema.
 - *Attributi presunti:* idUtente (univoco), username, password (eventualmente gestita con hashing), ruolo (es. Admin vs Cliente standard).
- **Proiezione.java:** L'unità fondamentale del cinema.
 - *Attributi presunti:* idProiezione, titoloFilm, sala (es. Sala 1), orarioProiezione (tipo LocalDateTime), capienzaTotale, postiDisponibili.
- **Prenotazione.java:** Classe associativa che risolve la relazione Molti-a-Molti tra Utenti e Proiezioni.
 - *Attributi presunti:* idPrenotazione, idUtente, idProiezione, numeroPostiPrenotati.

5. Logica di Business: I Gestori (Core Classes)

I tre file Gestore rappresentano il "motore" applicativo.

GestoreProiezioni.java

- **Ruolo:** Inizializza il catalogo leggendo proiezioni.csv.
- **Funzionalità:** Implementa metodi di filtro. Ad esempio, una funzione `cercaPerTitolo(String titolo)` itererà sulla lista in memoria restituendo un sottoinsieme di proiezioni. Mantiene aggiornato il conteggio dei `postiDisponibili` per evitare l'overbooking.

GestoreUtenti.java

- **Ruolo:** Gestisce il ciclo vitale dell'account leggendo e scrivendo su `Utenti.csv`.
- **Funzionalità:**
 - *Login:* Confronta le credenziali inserite dall'utente con quelle caricate in memoria.
 - *Registrazione:* Verifica l'assenza di duplicati (username già esistenti) prima di eseguire una procedura di *append* sul file CSV.

GestorePrenotazioni.java

- **Ruolo:** Coordina Utente e Proiezione.
- **Flusso di esecuzione di una prenotazione:**
 1. Verifica tramite `GestoreProiezioni` che la proiezione esista e che i `postiDisponibili` siano $\geq$ ai posti richiesti.
 2. Se il controllo passa, genera una nuova istanza di `Prenotazione`.
 3. Aggiorna in memoria i posti residui del film.
 4. Salva la transazione in modalità *append* su `Prenotazioni.csv`.

6. Gestione della Persistenza e Operazioni di I/O

Il sistema utilizza tre file CSV il cui formato prevede l'utilizzo della virgola (,) o del punto e virgola (;) come delimitatore standard.

Approccio di Lettura (Parsing):

I file vengono letti tramite `BufferedReader`. Ogni riga viene processata utilizzando il metodo `String.split(delimitatore)`. I dati estratti (inizialmente stringhe) vengono convertiti nel tipo nativo corrispondente (es. `Integer.parseInt()` per i posti, `LocalDateTime.parse()` per le date) e iniettati nei costruttori delle Entità.

Approccio di Scrittura:

- **Aggiunta (Append):** Per registrare un nuovo utente o una nuova prenotazione, viene aperto un `FileWriter` con il flag `append = true`. Questa operazione è estremamente veloce e sicura, non rischiando di sovrascrivere dati preesistenti.
- **Modifica/Cancellazione:** Se il sistema prevede l'annullamento di una prenotazione, viene utilizzata una strategia **Copy-Swap**: si legge il file riga per riga, si scrivono su un file temporaneo tutte le righe tranne quella da eliminare, si elimina l'originale e si rinomina il file temporaneo.

7. Gestione delle Eccezioni e Robustezza

Un'applicazione CLI che interagisce costantemente con il File System è soggetta a potenziali crash. CineMax deve implementare una solida gestione del blocco try-catch:

- `FileNotFoundException` / `IOException`: Catturate e gestite elegantemente qualora i file CSV non esistano (ad esempio, al primo avvio). Il sistema, nei Gestori, dovrebbe creare file vuoti con gli header (le intestazioni delle colonne) per inizializzare l'ambiente.
- `InputMismatchException` / `NumberFormatException`: Intercettate nel loop di `CineMax.java` qualora l'utente inserisca testo dove è atteso un numero (es. la scelta di un menu o il numero di posti da prenotare).

La classe di test `testMain.java` suggerisce l'adozione di un approccio metodico per simulare queste eccezioni senza avviare l'intero flusso dell'interfaccia utente.

8. Analisi Algoritmica e Strutture Dati

L'uso oculato delle strutture dati assicura alte prestazioni.

- **Tempo di Caricamento (Startup):** $O(U) + O(P) + O(R)$, dove U, P e R sono rispettivamente il numero di Utenti, Proiezioni e Prenotazioni(Reservations) nei file CSV. Essendo una lettura sequenziale, il costo è puramente lineare.
- **Occupazione di Memoria (Space Complexity):** L'intero database viene mappato in collezioni in RAM (es. `List<Proiezione>`). Questa scelta ($O(N)$ spaziale) è ottimale per i gestionali di dimensioni medio-piccole, in quanto permette ricerche a velocità elevatissima senza dover riaprire il file su disco.
- **Complessità di Prenotazione:** Aggiungere una prenotazione richiede un tempo costante in scrittura $O(1)$ su file, garantendo che il sistema non rallenti all'aumentare dello storico delle prenotazioni.

Se questa estensione tecnica è completa per le tue esigenze, posso procedere immediatamente con la redazione del **Manuale Utente per CineMax**, dove spiegherò nel dettaglio (passo dopo passo) come l'utente o l'amministratore interagiscono concretamente con i menu testuali!